Left Behind, Not Forgotten: Reusing Regression Tests to Find Bugs in WebAssembly Runtimes

Ishrak Hayet, Mohsen EsfandyariDoulabi, Giorgi Merabishvili, and Marcelo d'Amorim

***Abstract*—WebAssembly (WASM) is gaining traction as a portable, high-performance, and secure execution technology for its ability to deliver near-native speed across platforms. WASM runtimes are optimized systems capable of executing binary WASM modules. As complex software, WASM runtimes contain bugs. Fuzzing is a popular approach to finding bugs in these systems, but an empirical study involving three popular WebAssembly runtimes, existing fuzzers, and existing seed corpora suggests that the bug-revealing ability of fuzzers is increasingly constrained by seed corpora that fail to keep pace with the evolving WebAssembly ecosystem.**

This paper reports on a large-scale empirical study of regression test reuse for WebAssembly runtimes. We observe that independently developed runtimes adopt standardized WebAssembly proposals asynchronously, causing semantic corner cases discovered in one runtime to remain relevant to others. Motivated by this observation, we investigate two complementary forms of regression test reuse: (1) transplanting regression tests across runtimes and (2) using regression tests as seed corpora for fuzzing. Our study revealed 36 previously unknown bugs across three WASM runtimes, including 13 through transplantation and 23 through fuzzing, substantially exceeding the number of bugs found using well-known seed corpora. Our findings provide empirical evidence that regression tests constitute an effective testing resource for independently evolving WebAssembly runtimes and yield actionable recommendations for future testing.

***Index Terms*—WebAssembly, Fuzzing, Seed Corpora Evaluation, Differential Testing**

I. Introduction

WebAssembly (WASM) is emerging as a portable, high-performance, and secure execution technology. Originally designed for web browsers, it has evolved into a general-purpose compilation target for languages such as C, C++, and Rust. WASM delivers near-native speed while providing strong security guarantees and consistent behavior across platforms. WASM runtimes (e.g., WASMTIME [1], WASMER [2], WAMR [3], and WASMEDGE [4]) are responsible for loading, verifying, and executing WASM binaries. These systems are highly optimized and widely deployed, powering applications that range from serverless and edge computing to embedded systems. However, like any complex software, WASM runtimes are prone to bugs. Their components such as interpreters, JIT compilers, and sandboxing layers can harbor subtle correctness, performance, and security issues. Such bugs may lead to crashes, miscompilations, or vulnerabilities. As WASM adoption grows, ensuring the robustness and correctness of its runtimes becomes critical. Developing effective methods to uncover and fix bugs in WASM runtimes is therefore an important research challenge.

Several fuzzers have been proposed to detect bugs in WASM runtimes, e.g., WADIFF [5], WAPPLIQUE [6], WASMAKER [7], and LWDIFF [8]. These tools have previously uncovered many inconsistencies, crashes, and specification deviations across runtime implementations. However, no prior work has studied the continued effectiveness of these fuzzers when used with the existing seed corpora. As observed across various domains [9]–[13], fuzzing effectiveness heavily depends on the quality and diversity of the input corpus. The quality and diversity of an outdated corpus may degrade over time as its effectiveness becomes saturated; thereby leading to limited coverage and fewer bugs found, whereas a well-curated corpus from a new source can substantially extend the effectiveness and relevance of existing fuzzers. Understanding how corpus quality influences WASM fuzzing outcomes remains a crucial step toward more systematic and effective testing of WASM runtimes.

With the goal of evaluating the performance of WASM fuzzers on three established seed corpora from the literature, we conducted a preliminary investigation involving three WASM runtimes and four fuzzers. Each fuzzing campaign was executed for 24 hours and repeated five times. The results revealed that the number of newly found bugs is substantially lower than that reported initially by these fuzzers on the same corpora. This pattern reflects the well-documented "saturation effect" in fuzzing [14], [15]: as targets mature, fixed seed corpora reveal fewer new bugs. A similar dynamic appears in compiler fuzzing, where bug discovery spikes after major version transitions and then declines [16]–[18]. These observations motivated us to examine alternate sources of seeds that might be more resilient to saturation.

Bugs in software systems tend to concentrate according to the Pareto distribution [19], which is why repeated bug fixes are usually spatially localized [20]. Intuitively, regression tests (i.e., that previously revealed bugs) are rich in features, they have high potential to reveal new bugs. Based on this conjecture, **this paper presents a large-scale empirical study on how regression tests can be used to reveal new bugs in WASM runtimes**. Specifically, we examine the effectiveness of two complementary approaches leveraging regression tests to find bugs in WASM runtimes: (1) *test transplantation*, where regression tests are executed across different runtimes, and (2) *fuzzing* with a seed corpus of regression tests. Although regression tests have been used before for test generation [21], [22], we are the first to systematically study the effectiveness of regression tests to find new bugs in the WebAssembly

Ishrak, Mohsen, Giorgi, and Marcelo are with North Carolina State University, Raleigh, NC 27695 USA. (e-mail: {ihayet, mesfand, gmerabi, mdamori}@ncsu.edu)

domain using both test transplantation and fuzzing. We hypothesize that regression tests are particularly valuable in the WebAssembly ecosystem because independent runtimes adopt new specification proposals asynchronously, allowing semantic corner cases captured in one implementation to remain effective in exposing analogous defects in others.

To support our study, we introduce RTBENCH, a dataset of 584 WebAssembly regression tests collected from multiple runtimes that we use for transplantation and as seeds for fuzzing. Our results show that the two bug-finding mechanisms we employ are complementary in revealing bugs in WASM runtimes. Transplanting regression tests uncovered **13** previously unknown bugs which is a modest but significant number, when compared to the 10 bugs found by the dedicated fuzzer WAPPLIQUE after several hours of fuzzing. Using RTBENCH as a seed corpus markedly improved fuzzing effectiveness: fuzzers achieved higher coverage and exposed **23** additional bugs compared to three existing seed corpora used in the literature. Importantly, none of the bugs discovered through transplantation overlapped with those found by fuzzing, highlighting their complementary nature. The detected issues spanned compilation and runtime failures, including crashes and output mismatches. Notably, the most effective seeds originated from regression tests created two to three years ago, coinciding with the transition from WebAssembly version 1 to 2, while older tests, though less useful for fuzzing, continued to reveal unique issues through transplantation.

This paper makes the following contributions:

- ⋆ An in-depth empirical study of using regression tests from existing WASM runtimes to identify bugs through complementary test transplantation and fuzzing;
- ⋆ A benchmark study of three established WASM seed corpora from the literature as a baseline for comparison;
- ⋆ RTBENCH - a dataset of over five hundred real-world WebAssembly regression tests collected from the issue trackers and regression suites of major runtimes: Wasmtime [1], Wasmer [2], WAMR [3], and WasmEdge [4].
- ⋆ **36** total bugs found across 3 WASM runtimes, out of which, **34** were newly reported. Our results are publicly available at https://github.com/ncsu-swat/fuzzing-wasm-study.

II. BACKGROUND

This section outlines background for the rest of the paper.

A. WebAssembly Module

WebAssembly (WASM) is a virtual instruction set architecture that is often used as a compilation target for various high-level languages. A WASM module is the fundamental unit of deployment and execution; it is a platform-independent binary containing functions, type definitions, and data. WebAssembly Text (WAT) format [23] is a textual representation of a WASM file. Decompilers from WASM to WAT exist [24], [25].

Figure 1 shows a recursive version of the factorial function implemented in C and its corresponding WAT code.

```c
// Factorial in C
long long factorial(long
    long n) {
  if (n == 0) {
    return 1;
  } else {
    return n * factorial(n
     - 1);
  }
}
```

(a) C code.

```
(module
  (type (func (param i64) (
    result i64))) ;; Type
    section
  (func (type 0) (param i64
    ) (result i64)
    local.get 0
    i64.eqz
    if (result i64) (
     i64.const 1) else
      local.get 0
      local.get 0
      i64.const 1
      i64.sub
      call 0
      i64.mul
    end)
  (export "factorial" (func
    0)) ;; Export section
)
```

(b) Corresponding WAT code.

Fig. 1: Factorial example.

The code snippet defines a standard recursive factorial routine which is compiled to an equivalent WASM module using the `clang` compiler. Figure 1(b) shows the different sections in the WAT format: the `Type` section defines type signatures (parameter and result types) on line 2; `Code` section contains the function bodies (locals and instructions) on line 3; and `Export` section assigns a public name '`factorial`' to the function on line 14, so that it can be externally invoked. The WASM code section is of particular importance to fuzzing as it contains the function bodies that collectively determine module behavior. Each instruction in WebAssembly consists of an opcode accompanied by zero or more parameters and operands. Given that a WASM module is executed within a stack-based virtual machine, instructions operate on an implicit operand stack rather than explicit registers. The execution of each instruction modifies the stack state according to its defined semantics, ensuring that the side effects and resulting values conform to the instruction's type signature and parameter specification.

B. WebAssembly Runtimes

A WebAssembly runtime is a stack-based virtual machine that provides an execution model to load, instantiate, and run programs that have been compiled as WASM binary modules [26]. A WASM runtime processes a module in three stages: (1) decoding, (2) validation, and (3) execution. Similar to a compiler, decoding converts the binary into an intermediate representation or directly into machine code. Validation ensures consistency of the types and the operand stack [27]. Execution takes place by directly invoking the `start` function, if the module defines one, or by loading the module, possibly from a different programming language or from the Command Line Interface (`CLI`) and calling exported functions.

A WebAssembly runtime maintains three stacks: an operand stack to track instruction operands, a control stack to track control flow, and an initialization stack to track local variables of a function [28]. There are two ways to execute a WASM module: (1) by executing directly on standalone WebAssembly runtimes that run on an Operating System [26] and (2) by embedding a WASM module inside a host application (e.g. a browser) with the necessary glue code and programmatic API access to the WebAssembly runtimes. This paper focuses on the standalone versions of the WebAssembly runtimes where we directly use their `CLI` to execute WASM modules.

C. Mutational Fuzzing of WASM Runtimes

Several domain-specific fuzzers have recently been proposed to assess the robustness and security of WASM runtimes [5]–[8], [29]. Most of these fuzzers are *mutational* [5]–[8], i.e., they generate new inputs through systematic mutations of existing inputs as opposed to creating new inputs. Typically, a mutational fuzzer initializes a population with a *seed corpus* and iteratively applies mutation operators to individuals from the population. In WASM mutational fuzzing, mutation operators can be applied at a low- or high-level data representation. More precisely, mutation operators can either modify WASM binary files or their corresponding abstract syntax trees (ASTs). For determining test outcomes, fuzzers typically rely on oracles that identify crashes or differential output among multiple runtimes.

As observed across multiple software domains, the quality and diversity of the seed corpus play a crucial role in determining the effectiveness of fuzzing [9]–[13].

III. WORKFLOW

Figure 2 illustrates the workflow that we use to find bugs in WASM runtimes. First, we extract (1) WASM test cases from GitHub issues of different runtime repositories and add them into RTBENCH (2). These mined test cases are then transplanted (3a) across runtimes (4) and also used separately as seeds for fuzzing (3b). All `.wasm` test cases (5) are then evaluated using differential testing (6) to identify behavioral inconsistencies across runtimes. Detected anomalies are filtered (6) to remove false positives and duplicates, then triaged (7) to produce bug reports (8).

A. RTBENCH

We propose RTBENCH, a seed corpus of WASM files mined from bug histories and regression tests. To construct our corpus, we collected seeds from real-world bug reports spanning a five-year period from 2020 to 2025. We mined the public GitHub issue trackers and regression testsuites of the four major WebAssembly runtimes, namely WASMTIME [1], WASMER [2], WAMR [3], and WASMEDGE [4] to collect 584 WASM files. The distribution of the collected seed testcases are shown in Table I.

TABLE I: Distribution of seeds in RTBENCH.

Runtime	Issues	Test-suite	Total
WASMTIME	33	-	33
WASMER	15	274	289
WAMR	106	126	232
WASMEDGE	30	-	30
Total	184	400	584

We focus on issues that refer to either crashing inputs or the ones leading to differential output. For each candidate, we first identify the buggy runtime version from the issue report. We then execute the test case on that version to confirm bug reproducibility. Importantly, we also verify that none of these seeds reproduce crashes on the latest versions of the corresponding runtimes at the time of our study, confirming that the underlying bugs have been fixed. This approach provided us with a seed corpus grounded in actual historical bugs while ensuring they had indeed been buggy in the reported versions and that they no longer manifest in current releases. These checks help to form a realistic and reliable seed corpus for subsequent fuzzing and cross-runtime differential testing.

When collecting test cases from the runtime testsuites, we only select the `.wasm` files since the fuzzers expect only `.wasm` files as seeds (§II-C). For testcases collected from issues, we only keep the ones related to `CLI` bugs of the runtimes (§II-B). For testcases collected from both issues and testsuites, we filter out duplicates and non-deterministic cases.

B. Fuzzer Provenance Logging

For any large-scale fuzzing campaign, it is essential not only to generate diverse test cases but also to maintain a clear record of how those test cases were derived. As discussed at Section II-C, the two mutation-based fuzzers used in our study, WAPPLIQUE [6] and WASMAKER [7], generate new WASM modules by splicing one or more existing WASM modules. To ensure reproducibility and accountability across this process, we extended both fuzzers with a lightweight provenance logging mechanism that records the original seeds, the fragments used, and the mutation operations applied.

In WASMAKER [7], seed WASM binaries are parsed into per-function abstract syntax trees (ASTs), analyzed at the opcode level, and then mutated to form new inputs. We integrated provenance logging at the AST generation stage. Similarly, WAPPLIQUE [6] decomposes real WASM seeds into reusable code fragments from function bodies, which are then recombined and slightly mutated to synthesize new test cases. To ensure that these instrumentation changes did not alter fuzzer correctness or efficiency, we validated the modified fuzzers against their original behavior. Both fuzzers produced the same ratio of valid to invalid test cases as their uninstrumented versions. The overhead of provenance logging was negligible This validation confirmed that our logging mechanism maintained functional equivalence while adding transparent, low-cost traceability to our fuzzing campaigns.

C. Differential testing

To systematically uncover inconsistencies across WebAssembly runtimes, we developed a large-scale differential testing framework that executes the same input WASM file under multiple runtime configurations and compares their outcomes. When running each input WASM file, we invoke all of the exported functions on different modes of the runtimes.

1) Runtime Modes.: Our differential testing framework supports a variety of execution and compilation modes across the tested runtimes. For WASMTIME, both direct interpretation and execution of precompiled (`.cwasm`) artifacts are tested. WASMER is evaluated under its `Cranelift` and `LLVM` backends. WAMR is exercised through its ahead-of-time compiler `wamrc`, native execution in AOT mode, interpreter, and JIT mode using the `iwasm` application. WASMEDGE is tested in JIT mode and interpreter mode with the `wasmedge` application, and compiled mode with `wasmedgec` compiler. Each run is wrapped within a 10-second timeout to detect hangs, ensuring the module can capture various types of runtime behaviors while avoiding indefinite stalls. This layered design provides a robust methodology for discovering cross-runtime semantic mismatches and exposing previously unknown bugs.

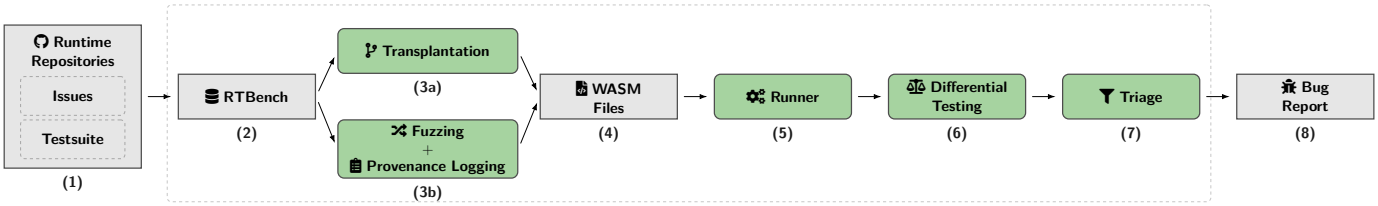


Fig. 2: Workflow of Study Design. Green rectangles represent processes, while grey rectangles denote data or artifacts.

TABLE II: Seed corpora used in our evaluation, their count, binary size, control-flow nesting depth, and baseline coverage achieved across four WebAssembly runtimes when replayed directly without fuzzing.

Seed Corpus	Count (K)	Size (KB)		Nesting Depth (K)		Branch Coverage (%)			
		Mean	SD	Mean	SD	Wasmtime	Wasmer	WAMR	WasmEdge
LLVMBENCH	0.22	15	8.80	0.22	0.12	11.02	12.02	14.39	27.53
RTBENCH	0.58	584	3900	2.95	1.97	13.73	16.12	18.23	28.57
WASMBENCH	8.50	700	3200	1.86	1.48	12.31	11.97	17.71	27.67
SPECBENCH	200	0.27	0.13	0.00	0.00	10.89	10.01	15.32	19.81

2) Output Normalization.: To minimize false positives and capture semantic output differences instead of just cosmetic divergences, we have developed a postprocessing layer which normalizes runtime outputs and maps diverse error messages to standardized categories. Each runtime (WASMTIME, WASMER, WAMR, and WASMEDGE) has a dedicated postprocessor function that detects recurring failure modes such as `out_of_bounds`, `integer_divide_by_zero`, `invalid_func_nargs`, or `unsupported`. Outputs that involve floating-point are normalized into consistent mantissa–exponent representations, and +/- 'NaN'/'nan' literals are standardized to ensure comparability. By filtering away runtime-specific warnings, trimming extraneous logs, and unifying divergent error messages, the module isolates semantically meaningful differences. This guarantees that any observed discrepancies across runtimes reflect true semantic divergences rather than cosmetic variations in error reporting.

D. Triaging

After identifying likely anomalies, we triage these cases by deduplication and simplification which are elaborated below.

1) Deduplication: To efficiently triage recurring crashes across WebAssembly runtimes, we implemented a crash stack trace deduplication module that couples a Python callback in LLDB [30] with a Bash wrapper for automated execution. The wrapper script launches the program under LLDB in batch mode, installs a custom crash hook, and configures signal handling for `SIGSEGV` and `SIGABRT`. For Rust-based runtimes, we configured the panic mode to abort. When such abnormal terminations occur, the Python hook records the top five frames of the crashing thread, including function names and source file locations, filtered by a configurable path pattern. These frames are appended to a trace file and act as a concise fingerprint of the failure. By grouping test cases with identical top-frame sequences, the module coarsely clusters crashes that share the same underlying defect while avoiding over-fragmentation due to deeper stack variations. This design reduces redundant triage effort by collapsing large sets of failing test cases into representative clusters that can be more thoroughly inspected.

To triage differential outputs observed across runtimes, we built an execution tracing module that conditions on the divergent output itself. The module leverages LLDB to perform instruction-level tracing of a target program until a specified output string appears on stdout. When a mismatch is detected between runtimes, the expected differential output is supplied as the tracing target, and execution halts as soon as this output is observed. At each step, the tracer records the current function, file, and line number, producing a detailed trace of the program's execution path leading to the discrepancy. Since such traces can be verbose and include incidental logging, we focus on the most recent fifteen frames preceding the divergent output as a fingerprint for deduplication. By comparing these condensed trace signatures, the module distinguishes whether two test cases that generate the same output pattern actually expose the same underlying bug or separate issues. This approach improves the efficiency of triaging differential outputs.

2) Test Reduction: Before reporting bugs, we apply automated test case reduction to minimize failing inputs while preserving the triggering behavior. This is handled through a reducer module built around the `wasm-tools shrink` utility, which iteratively simplifies WebAssembly binaries under the guidance of a custom predicate. The predicate runs the reduced module through our differential test engine and deduplication pipeline, compares the produced outputs against the reference failing outputs, and accepts the minimized candidate only if the failure is preserved. This ensures that irrelevant instructions and sections are pruned away, leaving behind the smallest input that still reproduces the crash or differential output. The module produces both a minimized `.wasm` file and a corresponding textual `.wat` representation for inspection.

By combining various deduplication strategies with a systematic reduction, we prepared the final list of bugs and test cases for reporting to the corresponding runtime issue trackers.

IV. OBJECTS AND QUESTIONS

We elaborate the objects we use in the study and the research questions we pose in the following.

A. Objects

Seed corpora. Table II provides a high-level comparison of the seed corpora used in our evaluation. We evaluate three input corpora in addition to our regression-based corpus (RTBENCH): LLVMBENCH [6], WASMBENCH [7], [31], and SPECBENCH [5]. The other three corpora are publicly-available WASM datasets used in prior research.

In Table II, we show the mean testcase size and control-flow nesting depth as proxy metrics for testcase complexity. We note that SPECBENCH seeds have the lowest size and nesting depth because they are simple and focused seeds derived from the WASM specification [5]. In contrast, LLVMBENCH, WASMBENCH, and RTBENCH are relatively more complex and diverse corpora in terms of size and depth. RTBENCH has fewer seeds than most and they exhibit the highest standard deviation in both size and nesting depth, indicating that RTBENCH is highly diverse. Additionally, we observe that RTBENCH achieves higher initial coverage compared to the other corpora.

WASM runtimes. We target three runtimes: WASMER (20K⋆)(v5.0.6) [2], WASMEDGE (10K⋆)(v0.15.0) [4], and WAMR (6K⋆)(v2.4.1) [3], which are under active development and are highly popular. We also ran all the experiments on WASMTIME runtime, but since we did not find any bugs in it, we exclude it from the tables related to bug counts.

WASM fuzzers. We use four mutational WASM fuzzers: WADIFF [5], WASMAKER [7], WAPPLIQUE [6], and LWDIFF [8]. We did not consider general-purpose fuzzers, as libFuzzer [32] or AFL++ [33], because prior work has shown that customized fuzzers are significantly more effective in the WASM domain compared to general-purpose fuzzers [6], [7], [34]. We also have not considered purely generative fuzzers like WASMSMITH [29] since it does not need a seed corpora.

Following the standard for campaign-durations [35], [36], we ran each fuzzing campaign for 24 clock-hours. For a fair comparison, we ran each campaign with the same resources on an x86_64 AMD EPYC 9684X server to ensure that our cpu-hours would exceed that of the prior work on WASM fuzzing [5]–[7]. Moreover, to account for randomness, we repeated each campaign five times. We have used the `llvm-cov` tool to measure coverage across all the runtimes.

B. Research Questions

We pose the following research questions related to RTBENCH:

1) **RQ1: How does seed corpus choice affect fuzzing effectiveness across WASM runtimes?**
2) **RQ2: How effective is test transplantation for finding new bugs?**
3) **RQ3: How does transplantation compare to fuzzing?**
4) **RQ4: What are the characteristics of the bugs found?**
5) **RQ5: When do relevant seeds originate?**

The first question checks how well fuzzing finds new bugs when using different seed corpora and how seed choice affects coverage and bug discovery. The second question tests whether regression tests from one runtime can directly find bugs in others, showing how readily reusable these tests are across runtimes. The third question compares transplantation and fuzzing with respect to bug-finding ability. The fourth question examines what kinds of bugs appear and what parts of the runtimes they affect. Finally, the fifth question looks at when the effective seeds were created to learn whether older or newer tests are more useful for finding bugs today.

TABLE III: Number of unique bugs revealed by each combination of seed corpus, fuzzer, and runtime, aggregated over five 24-hour fuzzing campaigns. Column Σ aggregates unique bugs per corpus across fuzzers and runtimes. Bottom row aggregates per runtime across corpora and fuzzers. WASMTIME is excluded since no technique found a bug in it.

Seed Corpus	Fuzzer	WASMER	WAMR	WASMEDGE	Σ
LLVMBENCH	WADIFF	-	-	-	7
	WASMAKER	1	2	-	
	WAPPLIQUE	2	3	-	
	LWDIFF	-	-	-	
WASMBENCH	WADIFF	-	-	-	6
	WASMAKER	2	1	-	
	WAPPLIQUE	-	1	1	
	LWDIFF	-	1	1	
SPECBENCH	WADIFF	2	2	5	9
	WASMAKER	-	-	-	
	WAPPLIQUE	-	-	-	
	LWDIFF	1	1	-	
RTBENCH	WADIFF	-	-	-	23
	WASMAKER	7	3	3	
	WAPPLIQUE	7	2	1	
	LWDIFF	4	2	1	

V. EVALUATION

To answer the research questions, we evaluate the bug counts and characterize a selected number of the bugs found. Additionally, we measure coverage and analyze the origination timeline of seeds.

A. *RQ1: How does seed corpus choice affect fuzzing effectiveness across WASM runtimes?*

We compare four seed corpora across four fuzzers and three runtimes: three established baselines from prior WASM fuzzing work [5]–[7]: LLVMBENCH, WASMBENCH, and SPECBENCH. The fourth corpus is RTBENCH, our regression-derived corpus.

Table III reports the unique bug counts for each corpus, fuzzer, and runtime combination. The three established corpora reveal 15 unique bugs combined. This is substantially lower than the numbers reported in their original publications. For example, WAPPLIQUE originally reported 20 bugs with LLVMBENCH [6], and WASMAKER originally reported 33 bugs with WASMBENCH [7]. This decline is consistent with the saturation effect discussed in Section I.

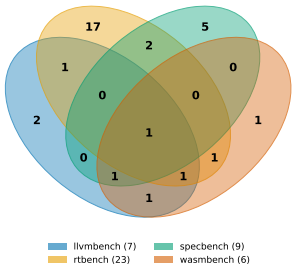


Fig. 3: Venn diagram showing fuzz bug membership of different seed corpora.

In contrast, RTBENCH alone reveals 17 unique bugs (ref. Figure 3) . This is more than any individual established corpus, and also more than the three established corpora combined. Figure 3 shows this as a Venn diagram of bug overlap

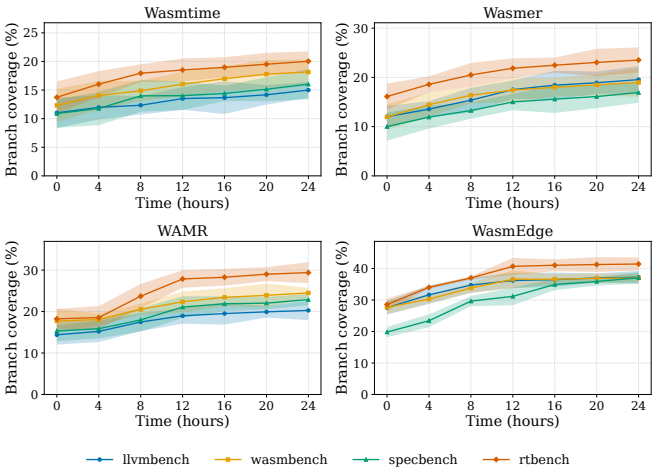


Fig. 4: Mean coverage progress with ±1 std. dev. when fuzzing with different seed corpora on all four runtimes.

```
(func (result externref)
  (local externref)
  local.get 0
)
```

(a) Bug-causing WAT code with no `externref` initialization.

```
(func (result externref)
  (local externref)
  ref.null extern
  local.set 0
  local.get 0
)
```

(b) Corresponding non-crashing WAT code with explicit initialization.

```
// WasmEdge/lib/executor/executor.cpp:164
auto *Inst = Val.get<RefVariant>().getPtr<Runtime::Instance
    ::CompositeBase>();
assuming(Inst);
```

(c) Crash site in executor.cpp

Fig. 6: Illustration of the **uninitialized externref bug** (WasmEdge#4325). The uninitialized local variable in (a) triggers a null pointer check in WASMEDGE (c). For comparison, the non-crashing WAT code is shown in (b). Confirmed and fixed with pull request (PR#4329).

across corpora. Table III shows the breakdown of bug counts across all corpora, fuzzers, and runtimes. The same pattern holds across runtimes. Only WASMAKER, WAPPLIQUE, and LWDIFF were effective with RTBENCH. WASMAKER and WAPPLIQUE found a comparable number of bugs. WADIFF did not yield bugs when paired with RTBENCH. We discuss this fuzzer-corpus mismatch in Section VI-A.

Figure 4 shows the coverage progress across all runtimes when using different fuzzers with all four seed corpora at 4 hour intervals. The shaded bands represent ±1 std. dev. around the mean. We observe that fuzzing improves overall coverage compared to the coverage achieved prior to fuzzing (i.e. at timestep 0) by 5-10%. While WASMEDGE and WAMR runtimes achieved relatively higher coverage, WASMTIME had the lowest coverage for all seed corpora. We also note that for all runtimes, fuzzing with RTBENCH consistently achieved higher coverage compared to the other seed corpora. Narrow bands around the mean depict an overall stable coverage measurement. Also, most fuzzers plateau around 12 hours, which aligns well with findings from prior work [37].

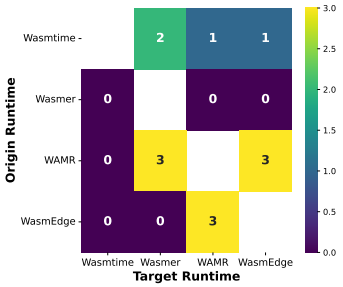


Fig. 5: Heatmap showing the number of bugs found from cross-runtime input transplantation using the RTBENCH seed corpora.

> ***Summary of RQ1.*** Using RTBENCH as a seed corpus significantly improved fuzzing results. The fuzzers we analyzed revealed 23 crashes/inconsistencies and achieved higher coverage when using RTBENCH compared to other corpora.

B. *RQ2: How effective is test transplantation for finding new bugs?*

This question examines the baseline effects of RTBENCH, that is, the benefits of using RTBENCH directly on each runtime. We first analyze the impact of directly executing regression tests from one runtime on another, a process we refer to as "test transplantation" (§ V-B1). We then present a longitudinal study of the tests that revealed bugs (§ V-B2).

1) RQ2.1: Can regression tests from one runtime directly reveal bugs in a different runtime?: Figure 5 shows a heatmap matrix to illustrate the results of running regression test cases across runtimes. The matrix represents the effects of running regression test cases from an origin runtime (y-axis) in a target runtime (x-axis).

We observe that regression tests from all runtimes except WASMER, could reveal new bugs in the other runtimes, but the effect is uneven. For example, WAMR-derived seeds were particularly cross-fertile, triggering 3 bugs in WASMER and 3 bugs in WASMEDGE. This case is highlighted with 2 yellow-colored squares in the row labeled "WAMR". Overall, these results indicate that test transplantation is capable of uncovering previously-hidden bugs in runtimes.

Figure 6 shows one such bug found in the WASMEDGE runtime by transplanting the test WAMR#3391. We have reduced the corresponding regression test for debugging and eventually reporting. Figure 6(a) shows the reduced code. The WAT code declares an `externref local` and immediately reads it with `local.get 0`, producing undefined data instead of a valid reference. During execution, the null value is interpreted, triggering a crash. To confirm whether this bug is indeed due to a lack of the necessary default initialization, we show a counterexample in Figure 6(b), where a similar WAT code that explicitly initializes `ref.null` does not trigger a crash. The WebAssembly Core Specification 4.2.1 requires that `externref` variables are initialized to `ref.null` [38]. Thus, the user is not responsible for explicitly initializing `externref` variables to `ref.null`. Hence, the absence of default initialization in WASMEDGE risked an eventual null pointer dereference. This bug was fixed by the pull request (PR#4329) shortly after reporting.

2) RQ2.2: When do bugs found by transplantation originate in the buggy runtimes?: Figure 7 shows the timelines

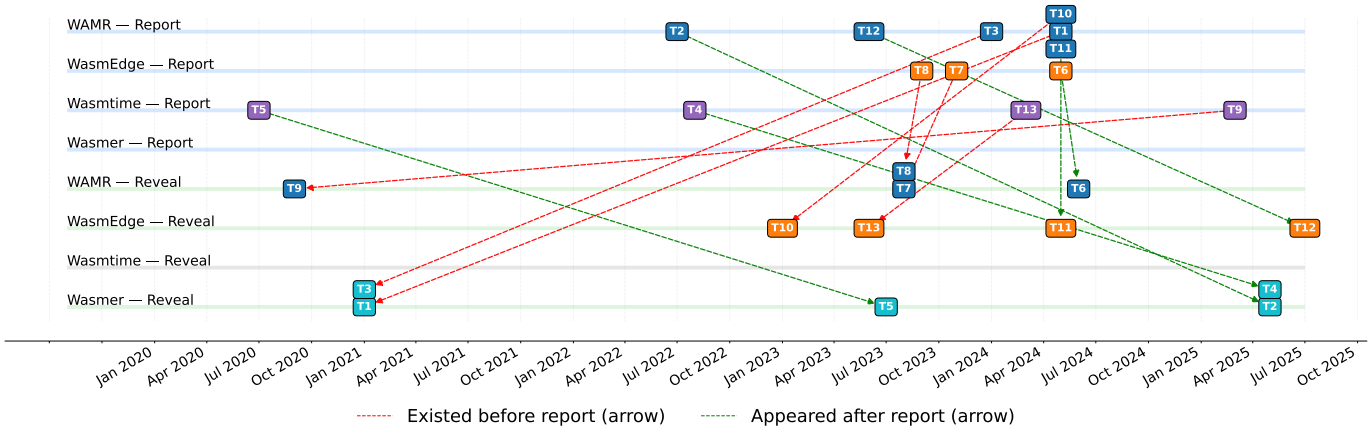


Fig. 7: Scatterplot showing timelines comparing dates of a buggy test case reported on one runtime and the revelation of another bug found with the same test case on another runtime. The small boxes show ids of specific test cases that were transplanted from one runtime to another. The directions of the arrows represent the runtime/timeline of a test case source and the corresponding runtime/timeline of the transplantation target.

of bugs revealed with transplantation as pairs of the form $\langle(\alpha, x, R_a), (\beta, x, R_b)\rangle$, where α denotes a point in time where a test case x *appears* in a bug report from runtime R_a and β denotes a point in time where x *reveals* a bug in runtime R_b. We report data quarterly to facilitate visualization. We use a binary search-like approach, similar to `git-bisect`, to identify the version of the buggy runtime that reveals the symptom for the first time.

The top-four horizontal lines from Figure 7 indicate the timelines associated with the origin of the test cases whereas the bottom-four horizontal lines indicate the timelines for revealing a bug with transplantation, i.e., the time a bug would be revealed for the first time in a different runtime. The boxes at both ends of each arrow show the ids of the test cases; the direction of the arrow (left or right) indicate if a bug existed in a runtime before it was reported in a different runtime. As we did not find any bugs on the WASMTIME runtime, the revelation bar of WASMTIME has been grayed out.

The visualization reveals two patterns. First, 7 of 13 bugs (i.e. 53.84%) existed long before their official reports (i.e., T1, T3, T7-10, and T13). Second, another 6 bugs (i.e. 46.15%) would only emerge in the observed runtimes long after they were originally reported elsewhere (e.g., T2, T4-6, T11-12). These patterns show that transplantation bugs exhibit substantial latency across runtimes: some bugs already exist, and other bugs will exist in the future. In both cases, transplantation of regression tests across runtimes is beneficial. The benefit would be immediate in the first case and would be delayed in the latter. Figure 7 also shows that most transplantation bugs cluster around 2022–2024, which was a period coinciding with major WebAssembly specification updates [39], [40].

Summary of RQ2. Regression test transplantation revealed a modest but important number of bugs in WASM runtimes: **13**, overall. For reference, the fuzzer WAPPLIQUE needed several hours of fuzzing to reveal 10 bugs in WASM runtimes.

TABLE IV: Status of reported bugs found using RTBENCH. The table shows transplantation bugs, fuzz bugs, and their totals. R=Reported, C=Confirmed, and F=Fixed. *Out of total 36 crashes/output differences, 2 unique crashes later found by LWDIFF in the WAMR runtime were not reported because they are fixed on the latest version of the runtime.

	Transpl. Bugs			Fuzz Bugs			Total Bugs		
Runtimes	**R**	**C**	**F**	**R**	**C**	**F**	**R**	**C**	**F**
WASMER	5	3	3	13	11	6	18	14	9
WAMR	4	3	0	4	3	0	8	6	0
WASMEDGE	4	4	3	4	3	2	8	7	5
Total	13	10	6	21	17	8	34*	27	14

C. *RQ3: How does transplantation compare to fuzzing?*

Table IV summarizes the status of bugs discovered by RTBENCH, divided into transplantation bugs, fuzz bugs, and their combined totals across four major WebAssembly runtimes: WASMTIME, WASMER, WAMR, and WASMEDGE. Several important observations emerge. First, fuzzing produced a substantially higher number of reported bugs compared to transplantation, indicating that fuzzing remains the more prolific bug-finding strategy overall. However, transplantation still uncovered unique bugs, particularly in WAMR and WASMEDGE. Second, WASMER had the largest number of total reported and confirmed bugs. Third, WASMEDGE and WASMER are the only runtimes with fixed bugs.

Since every fuzzing campaign begins by executing the seed corpus, bugs directly exposed by regression tests are naturally observed before mutation. To determine whether mutation independently rediscovered these bugs, we analyzed the provenance logs of every fuzz-generated test case that reproduced a transplantation bug. In every case, the bug-triggering lineage contained the same regression test that had already exposed the bug through direct transplantation. We found no instance in which the same underlying bug was rediscovered through a lineage originating from a different seed. Thus, while mutation generated many additional bugs, the transplantation bugs remained attributable to the original transplanted regression tests.

TABLE V: Categories, WASM proposal, and discovery technique for a sample of bugs we report with RTBENCH.

ID	Category	Proposal	Symptom	Found with	Error Conditions
1	Runtime Failure	Ref. Types	Different Output	Transplantation	`ref.null` operand is a typed function reference id.
2		Ref. Types	Crash	Transplantation	Uninitialized reference type used.
3		SIMD	Different Output	Fuzzing	Extracted signed lane used as memory load address.
4		SIMD	Different Output	Fuzzing	`i16x8.extract_lane_s` value used as `v128.load32x2_u` address.
5		SIMD	Crash	Fuzzing	Non-terminating recursive call in vector-returning function, followed by `unreachable`.
6		SIMD	Different Output	Fuzzing	Signed lane extraction ● used as effective address.
7	Compilation Failure	SIMD + Multi-value	Crash	Fuzzing	Function with return type `(i32, i32, v128)`.
8		SIMD	Crash	Fuzzing	NaN, then f32 SIMD op, then f64 SIMD op.
9		SIMD	Different Output	Fuzzing	SIMD constant ● `.ne` ● 4 SIMD cast ops.
10		SIMD	Crash	Fuzzing	`i32x4.lt_u` on a vector from `f64x2.lt`.
11		SIMD	Crash	Fuzzing	Lane-extracted i32 address used for `v128.load8_splat`.
12		Ref. Types/GC	Different Output	Transplantation	`ref eq` used as a function-type parameter.
13		Core/module	Different Output	Transplantation	Remove all functions.

```
(module...
  (func (;0;) (type 0) (result i32 i32 v128)
    i32.const 0
    i32.const 0
    v128.const i32x4 0x00000000 0x00000000 0x00000000 0
     x00000000
  )
)
```

(a) Compilation bug related to SIMD (Wasmer#5667).

```
(module
  (type (;0;) (func))...
   (func (;0;)
     br 0 ref.null 0
     unreachable
   )
)
```

(b) Runtime bug related to GC/RefTypes. (WAMR#4480)

```
(module
  (memory (;0;) 1 2)

  (data (;0;)
    (i32.const 896340315) ""
  )
)
```

(c) Runtime bug in core execution. (Wasmer#5734)

Fig. 8: Representative bugs revealed by RTBENCH.

TABLE VI: Characterizing the reported bugs by WASM proposal/subsystem and discovery technique.

Proposal / Subsystem	Transplantation	Fuzzing	Total
SIMD/Multi-value	2	12	14
Core execution engine	0	6	6
WASI / host interface	3	2	5
Reference Types / GC	3	0	3
Exception Handling	2	0	2
Core / module structure	1	0	1
Threads + CLI	1	0	1
Uncertain	1	1	2
Total	**13**	**21**	**34**

Summary of RQ3. Fuzzing generally found more total bugs, but transplantation found different ones. None of the bugs from transplantation overlapped with those from fuzzing. Transplantation complements fuzzing.

D. *RQ4: What are the characteristics of the bugs found?*

Table V characterizes representative bugs revealed with RTBENCH as the seed corpus. These bugs are categorized by using terminology consistent with prior WASM testing work [6], [7]. We group them into two main categories: `runtime failure` and `compilation failure`. Runtime failures refer to crashes that occur during execution, which we further distinguish by the type of exception. A compilation failure occurs when a Wasm binary cannot be successfully compiled or instantiated, i.e. it fails before execution begins.

We further categorize bugs based on their symptoms. Each testcase's output was analyzed to determine whether it belongs to the `Crash` or `Output-difference` category. If a testcase triggers a runtime crash (e.g., `segfault` or `aborted`), it is placed under the crash category; if it produces inconsistent outputs across runtimes, it is classified as an output-difference bug. The categorization process begins by minimizing each testcase with `wasm-tools shrink` to its smallest reproducible form, then identifying the specific code segment responsible for the behavior. To validate our classification, we modified and relocated the identified code segments to confirm that they consistently reproduced the observed behavior. Finally, the structure of each minimized code pattern guided its placement into the most relevant category.

For instance, one representative runtime failure (Row 5) involves a recursive call inside a vector-returning function that never terminates and is eventually followed by an `unreachable` instruction, leading the runtime into an infinite loop before crashing. Another case (Row 11) represents a compilation failure where the compiler miscomputes lane extraction as an `i32` address and then incorrectly uses this address as the base for a SIMD load `v128.load8_splat`, triggering a validation error during instantiation.

The following examples illustrate three test cases related to different kinds of failures. Each case highlights the key instruction patterns and runtime behaviors that led to the observed failures.

Figure 8(a) reveals a crash pattern linked to incorrect handling of multi-value function returns involving SIMD vector types. The critical lines are `(type (func (result i32 i32 v128)))` and the instruction sequence `i32.const 0` → `i32.const 0` → `v128.const i32x4 0 0 0 0`. Here, the function is declared to return two `i32` values followed by a `v128` vector. While this is a valid multi-value return signature under the WebAssembly SIMD and multi-value proposals, certain runtimes, particularly LLVM-based JIT backends mishandle the lowering of mixed scalar and SIMD return values. During compilation, the LLVM backend emits mismatched IR types between the declared function signature and the actual return instruction (e.g., returning `i64, i128` instead of the expected multi-value tuple), leading to an internal assertion failure "`Function return type does not match operand type of return inst`". This results in a compiler abort rather

than a clean trap. Therefore, the failure is classified as a `Compilation Failure type-mismatch bug`, specifically triggered by the combination of multi-value returns and SIMD vector types that cause inconsistencies in function return type lowering during LLVM IR generation.

Figure 8(b) demonstrates a runtime crash caused by incorrect handling of typed references within control-flow boundaries. The instruction sequence `br 0` → `ref.null 0` is responsible for triggering the failure. The `br 0` instruction transfers control to the nearest enclosing block, rendering subsequent code unreachable, yet the engine must still correctly validate or lower it. Immediately following this, the instruction `ref.null 0` attempts to create a null reference of heap type index `0`, which corresponds to the first function type defined in the module (`type (func)`). While this construct is syntactically valid under the reference-types proposal, certain runtimes particularly JIT-based ones mishandle the combination of an unreachable code region with a typed reference operand, leading to inconsistent type-state tracking or invalid IR generation during lowering. This results in a runtime failure, specifically in the runtime's implementation of reference-type validation and unreachable-block handling.

Figure 8(c) shows a core execution bug during runtime. The test case module has a 1-page memory (max 2) and an active data segment with empty contents at offset 896340315, far beyond the memory's size. Under bulk-memory semantics, an out-of-bounds active segment should result in a handled trap at instantiation even when empty. But in this case this test case resulted in an unhandled segmentation fault.

In Table VI, we characterized all reported bugs by proposal and by discovery technique. Two patterns stand out. All Reference-Types and GC bugs (3 of 3) were found only through transplantation. These bugs share a common cause: runtimes omit an implicit specification obligation, such as default-initializing an `externref` to `ref.null`, validating a typed-reference operand, or rejecting an illegal `ref eq` type constructor. A fuzzer's mutation operators rarely construct these cases, since the bug lies in an absent or misused construct rather than an executable instruction pattern. Conversely, all 6 bugs tied to the core execution engine rather than a specific proposal, including call-stack depth limits, trap and signal handling, and parser edge cases, were found only through fuzzing. SIMD is the largest category overall (14 bugs) but leans heavily toward fuzzing, which found 12 of them versus 2 from transplantation. WASI/host-interface bugs (5) were split between the techniques. These results show that transplantation and fuzzing (§V-C); target different classes of bugs.

Summary of RQ4. SIMD/Multi-value accounts for the largest share of bugs found with RTBENCH (14 of 36), predominantly via fuzzing (12 of 14). Reference-Types/GC bugs (3) were found exclusively through transplantation, while core-execution-engine bugs (6) were found exclusively through fuzzing, evidence that the two techniques target structurally different classes of bugs.

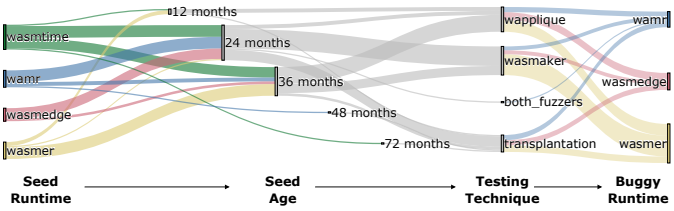


Fig. 9: Sankey diagram showing the lineage of bugs across WebAssembly runtimes. Flows connect the seed runtime, seed age, testing technique, and the buggy runtime where a new test case was observed. Source and target runtimes are color-coded consistently, while intermediate age and technique bands are shown in gray.

E. *RQ5: When do relevant seeds originate?*

Figure 9 highlights several important patterns in bug propagation from seeds to new test cases across different WebAssembly runtimes. First, seeds from WASMTIME and WAMR dominate the flow, producing a substantial number of downstream bugs, whereas WASMER and WASMEDGE appear more frequently as buggy runtimes than as primary seed sources.

Second, 90% of the impactful seeds are concentrated between the 12-month and 36-month age bands, a period that coincides with major implementation upgrades from WebAssembly specification version 1 to 2 [39], [40]. 14.63% of the seeds from this period (12-36 months old) were useful for finding bugs. In contrast, very recent seeds that are younger than 12 months, contribute less overall.

Third, the oldest seeds (48 and 72 months) were only effective when used with transplantation, underscoring that transplantation provides a unique pathway to reactivating older seeds that fuzzing alone cannot exploit. Only 6.86% of the seeds that were either $<$12 months or $>$36 months old were useful in bug finding.

Summary of RQ5. 90% of the useful seeds came from regression tests that were 2–3 years old, aligning with the period when WebAssembly's specification evolved from version 1 to 2. Older seeds, while less effective for fuzzing, still exposed new issues through transplantation.

VI. DISCUSSION

This section presents lessons learned and threats to validity.

A. *Lessons Learned*

We summarize key lessons learned as a set of observations and corresponding recommendations.

Reuse Regression Tests Directly on Asynchronously Evolving Implementations Although WebAssembly specifies standardized semantics, major language proposals (e.g., Reference Types, SIMD, Exception Handling, Memory64, and GC) are implemented by runtimes at different times depending on implementation priorities and available resources. Consequently, regression tests that expose corner cases in an early adopter frequently remain effective for revealing analogous implementation defects in later adopters. This asynchronous

proposal-adoption dynamic explains why regression tests remain valuable across independently developed runtimes and motivates the use of cross-runtime regression-test transplantation. This is borne out empirically: every Reference-Types/GC bug in our study was found through transplantation and none through fuzzing (§V-D), directly supporting this asynchronous-adoption hypothesis.

Monitor all runtimes and integrate new regression tests. Our study shows that regression tests from one runtime can directly reveal bugs in others without additional overhead and also serve as effective seeds for fuzzing (§V-A & V-B). Moreover, 53.84% of the bugs found by transplantation existed long before in the observed runtimes (§V-A). So, we recommend the integration of regression tests as soon as they are updated.

Prioritize regression tests created during major specification upgrades. We observed that 90% of the useful transplanted tests and fuzzing seeds originated during major specification upgrades from version 1 to 2 (§V-A & V-E) [39], [40]. Specification upgrades are routine and version 3 has been recently finalized [41]. Therefore, as implementations for version 3 start rolling out, we recommend automated test generation from the spec updates similar to prior work on spec-based test generation [42].

Continuously update and maintain RTBENCH. The long-term utility of RTBENCH depends on both its evolution and its curation. As new regression tests are introduced to fix bugs or cover new specification features, they should be promptly incorporated into RTBENCH to capture up-to-date runtime behavior. We also found that older seeds were largely ineffective for both transplantation and fuzzing (§V-E). So, older seeds exercising stable features or redundant paths should be periodically retired for the efficiency of RTBENCH seed corpora [43]–[45].

Adjust mutation strategy based on seed complexity. Our results indicate that mutation effectiveness of WASM fuzzers is tied to seed complexity. We observed that WADIFF's havoc mutations are effective on only SPECBENCH corpora (§V-B). It is worth noting that SPECBENCH contains simple and focused seeds (§IV-A). In contrast, WASMAKER and WAPPLIQUE's splicing mutations perform well only when applied to complex and diverse seed corpora such as RTBENCH (§V-B, §IV-A). Based on our observations, we recommend a seed-complexity aware mutation strategy i.e. to apply havoc mutations on simple seeds and splicing on complex ones for WASM fuzzing.

B. Case study: Why does RTBENCH overcome saturation?

To better understand why RTBENCH is able to overcome the saturation observed with existing seed corpora, we compared the line-level coverage of RTBENCH and WASMBENCH on the WASMER runtime. We selected this case because RTBENCH exhibits one of the largest initial coverage advantage over WASMBENCH in Table II. The comparison reveals that the additional coverage is not concentrated in simply instruction execution alone, but in runtime infrastructure responsible for instance management, VM context handling, memory management, trap handling, and runtime library calls. In particular, RTBENCH uniquely exercises code for memory initialization, copy, fill, growth, table initialization, data segment operations, imported and local memory resolution, and VM context manipulation, many of which remain completely uncovered when fuzzing starts from WASMBENCH.

These results suggest that RTBENCH's advantage stems from exposing fuzzing to diverse instructions and a broader set of runtime states and execution contexts, enabling mutations to continue exploring previously unreachable behaviors after conventional seed corpora have saturated.

C. Threats to Validity

Construct. A potential threat concerns how we define and count bugs. Some anomalies could stem from undefined runtime behaviors. Although our differential testing module normalizes outputs, nondeterministic effects may still yield spurious mismatches. To mitigate this, we identified patterns of known undefined behaviors and semi-automatically filtered such cases. Our proposal-level classification of the reported bugs (§V-D) was performed manually against each bug's minimized test case and error output; a second author independently reviewed the classification to reduce subjectivity, with disagreements resolved by discussion. **Internal.** Fuzzing inherently involves randomness, which can affect coverage and bug counts. To mitigate randomness bias, we executed each fuzzing campaign five times under fixed time budgets and identical configurations, then aggregated unique bug outcomes across runs. Additionally, we evaluate the corpus as a whole without showing exactly what properties of a seed drive gains. We partially mitigate this threat with a post-hoc mapping of bugs to WebAssembly proposals (Table VI). In both transplantation and fuzzing, we found that the presence of proposal-specific instructions is associated with effective seeds. We will analyze other seeds properties and their interaction in future work. **External.** Another threat relates to the representativeness of our objects of study i.e. runtimes, fuzzers, and seed corpora. We focused on four mature and four widely used mutational fuzzers, which may not generalize to emerging engines or generative fuzzers. Similarly, the RTBENCH corpus was derived from regression tests of these four runtimes and may underrepresent other WebAssembly ecosystems. We mitigate this by selecting the most widely used runtimes, fuzzers, and seed corpora from prior work, and by releasing RTBENCH to enable extension in future studies.

VII. Related Work

Regression Tests for Transplantation. Test transplantation reuses existing tests across systems by adapting harnesses, configurations, and oracles, and has proven effective for JavaScript engines, databases, and mobile applications [16]–[18], [46]–[53]. In JavaScript engines, regression tests are transplanted across engines to find new bugs [46], and in database systems, a single test suite is reused across SQLite, PostgreSQL, and DuckDB [47]. RTBENCH applies this idea to WASM by reusing regression tests from runtime repositories, confirming them on their original versions, and executing them across independent runtimes to detect divergences. Unlike

these settings, WASM is portable by design, so RTBENCH runs the same test cases on every runtime without any adaptation.

Regression Tests for Fuzzing. Regression tests can also serve as fuzzing seeds. Mutational fuzzers such as WAPPLIQUE, WASMAKER, WADIFF, and others [5]–[8], [34], [54], [55] transform existing modules to explore differential or buggy runtime behavior. RTBENCH builds on these by changing only the seed source, deriving seeds from real regression tests and bug reports. This exposes how much existing corpora have saturated: WAPPLIQUE and WASMAKER originally reported 20 and 33 bugs using LLVMBENCH and WASMBENCH [6], [7], yet the same fuzzers on the same corpora now reveal far fewer new bugs, consistent with the saturation effect as runtimes mature [14], [15]. Using RTBENCH instead reveals the most new bugs of any corpus we evaluate. Consistent with prior seed selection studies [44], [45], we also found that seed selection strongly influences fuzzing effectiveness. However, our study additionally shows that, in the WebAssembly domain, regression tests are an especially valuable seed source because they preserve proposal-specific semantic corner cases that continue to expose defects in independently and asynchronously developed runtimes.

Differential Testing of WebAssembly Runtimes. Differential testing runs the same module on several runtimes and treats output disagreements as bugs; WADIFF [5], WASMAKER [7], and WAPPLIQUE [6] follow this approach and serve as baseline fuzzers in our study. These prior work focus more on the fuzzing approach whereas we focus more on an empirical study of the seed corpora used by these fuzzers. Two recent techniques add LLM assistance: LWDIFF [8] supports differential testing of WebAssembly runtimes, and DrWASI [56] targets WebAssembly System Interface (WASI) implementations. LWDIFF uses an LLM to generate mutators from the WebAssembly specification [8]. DrWASI adapts behavior from corresponding POSIX APIs and generates WASM test cases that exclusively target only the WASI implementations [56]. Neither LWDIFF nor DrWASI reuses regression tests for fuzzing, whereas RTBENCH proposes a new regression tests based seed corpus for fuzzing WebAssembly runtimes. The LLM-based fuzzers and RTBENCH are complementary, and RTBENCH can seed an LLM-driven generator like LWDIFF.

Other Related Works. Other work targets WebAssembly reliability and fuzzing methodology more broadly. Structure and rule-guided generators such as WasmCFuzz, RGFuzz, and SwFuzz [57]–[59] build valid programs from a grammar or the specification. RTBENCH instead starts from real regression tests that already concentrate in bug-prone code, so the two are complementary as starting corpora. WasmChecker [60] detects type and memory errors through static analysis without fuzzing, while WBSan [61] provides sanitization to support binary-only fuzzing. WasmRef-Isabelle [62] provides a verified reference interpreter that can serve as a fuzzing oracle. The SoK on analysis techniques for WebAssembly [63] and work on random testing of WASI [64] survey this space. Studies on statistical rigor and reliable benchmarking [65]–[67] and platforms such as FuzzBench and GreenBench [35], [68] establish good practice for fuzzing evaluation. We follow this guidance by running repeated, fixed-budget campaigns and releasing our seeds. Mutation-analysis and mutator-level studies [69], [70] further motivate our use of default mutator settings across trials.

VIII. CONCLUSIONS

This paper presents RTBENCH, a dataset of real-world WASM regression tests, and reports on an in-depth evaluation of two complementary strategies (namely, test transplantation and regression-seeded fuzzing) for uncovering bugs in WASM runtimes. Across three major runtimes, we discovered 36 bugs, with transplantation and fuzzing revealing distinct and non-overlapping issues. Our results show that regression tests remain valuable long after their creation to assist bug-fixes, especially those created during major specification updates. Overall, results indicate that the use of project metadata, such as issues and release notes, is a promising direction to improve effectiveness of WASM fuzzers.

REFERENCES

[1] "Wasmtime," https://github.com/bytecodealliance/wasmtime, 2025, [Accessed 01-01-2025].

[2] "Wasmer," https://github.com/wasmerio/wasmer, 2025, [Accessed 01-01-2025].

[3] "WAMR," https://github.com/bytecodealliance/wasm-micro-runtime, 2025, [Accessed 01-01-2025].

[4] "WasmEdge," https://github.com/WasmEdge/WasmEdge, 2025, [Accessed 01-01-2025].

[5] S. Zhou, M. Jiang, W. Chen, H. Zhou, H. Wang, and X. Luo, "Wadiff: A differential testing framework for webassembly runtimes," in *ASE*. IEEE, 2023, pp. 939–950.

[6] W. Zhao, R. Zeng, and Y. Zhou, "Wapplique: Testing WebAssembly Runtime via Execution Context-aware Bytecode Mutation," in *ISSTA*, 2024, pp. 1035–1047.

[7] S. Cao, N. He, X. She, Y. Zhang, M. Zhang, and H. Wang, "Wasmaker: Differential Testing of Webassembly Runtimes via Semantic-aware Binary Generation," in *ISSTA*, 2024, pp. 1262–1273.

[8] S. Zhou, J. Wang, H. Ye, H. Zhou, C. Le Goues, and X. Luo, "Lwdiff: An llm-assisted differential testing framework for webassembly runtimes," in *ICSE*, 2025.

[9] B. P. Miller, L. Fredriksen, and B. So, "An empirical study of the reliability of unix utilities," *Communications of the ACM*, vol. 33, no. 12, pp. 32–44, 1990.

[10] M. Xu, S. Kashyap, H. Zhao, and T. Kim, "Krace: Data race fuzzing for kernel file systems," in *S&P*. IEEE, 2020, pp. 1643–1660.

[11] C. Lyu, S. Ji, Y. Li, J. Zhou, J. Chen, and J. Chen, "Smartseed: Smart seed generation for efficient fuzzing," *arXiv preprint arXiv:1807.02606*, 2018.

[12] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based greybox fuzzing as markov chain," pp. 1032–1043, 2016.

[13] V. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey," *TSE*, vol. 47, no. 11, pp. 2312–2331, 2021.

[14] A. Groce and J. Regehr, "The Saturation Effect in Fuzzing," https://blog.regehr.org/archives/1796, 2020, [Accessed 01-01-2025].

[15] D. Liyanage, M. Böhme, C. Tantithamthavorn, and S. Lipp, "Reachable Coverage: Estimating Saturation in Fuzzing," in *ICSE*. IEEE, 2023, pp. 371–383.

[16] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in c compilers," in *PLDI*, 2011, pp. 283–294.

[17] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," *ACM Sigplan Notices*, vol. 49, no. 6, pp. 216–226, 2014.

[18] V. Livinskii, D. Babokin, and J. Regehr, "Random testing for c and c++ compilers with yarpgen," *Proceedings of the ACM on Programming Languages*, vol. 4, no. OOPSLA, pp. 1–25, 2020.

[19] C.-Y. Huang, C.-S. Kuo, and S.-P. Luan, "Evaluation and application of bounded generalized pareto analysis to fault distributions in open source software," *TRE*, vol. 63, no. 1, pp. 309–319, 2013.

[20] R. Yue, N. Meng, and Q. Wang, "A characterization study of repeated bug fixes," in *ICSME*. IEEE, 2017, pp. 422–432.

[21] L. Plein, W. C. Ouédraogo, J. Klein, and T. F. Bissyandé, "Automatic generation of test cases based on bug reports: a feasibility study with large language models," in *ICSE Companion*, 2024, pp. 360–361.

[22] M. R. I. Rabin and M. A. Alipour, "Configuring test generators using bug reports: a case study of gcc compiler and csmith," in *SAC*, 2021, pp. 1750–1758.

[23] "WebAssembly Text Format," https://webassembly.github.io/spec/core/text/index.html, 2025, [Accessed 16-10-2025].

[24] "wasm2wat(1)," https://man.archlinux.org/man/wasm2wat.1.en, 2025, [Accessed 21-10-2025].

[25] "bytecodealliance/wasm-tools," https://github.com/bytecodealliance/wasm-tools?tab=readme-ov-file, 2025, [Accessed 21-10-2025].

[26] Y. Zhang, M. Liu, H. Wang, Y. Ma, G. Huang, and X. Liu, "Research on WebAssembly Runtimes: A Survey," *TOSEM*, 2025.

[27] "WebAssembly Overview," https://webassembly.github.io/spec/core/intro/overview.html#concepts, 2025, [Accessed 08-10-2025].

[28] "WebAssembly Validation Algorithm," https://webassembly.github.io/spec/core/appendix/algorithm.html, 2025, [Accessed 08-10-2025].

[29] bytecodealliance, "WASM-Smith," https://github.com/bytecodealliance/wasm-tools/tree/main/crates/wasm-smith, [Accessed 07-10-2025].

[30] "LLDB," https://lldb.llvm.org/, 2025, [Accessed 23-10-2025].

[31] A. Hilbig, D. Lehmann, and M. Pradel, "An Empirical Study of Real-world WebAssembly Binaries: Security, Languages, Use Cases," in *WWW*, 2021, pp. 2696–2708.

[32] "libFuzzer," https://llvm.org/docs/LibFuzzer.html, 2025, [Accessed 01-01-2025].

[33] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "AFL++: Combining incremental steps of fuzzing research," in *USENIX WOOT*. USENIX Assoc., Aug. 2020.

[34] L. Zhang, B. Zhao, J. Xu, P. Liu, Q. Xie, Y. Tian, J. Chen, and S. Ji, "Waltzz:{WebAssembly} runtime fuzzing with {Stack-Invariant} transformation," in *USENIX Security*, 2025, pp. 6159–6178.

[35] J. Metzman, L. Szekeres, L. Simon, R. Sprabery, and A. Arya, "Fuzzbench: An open fuzzer benchmarking platform and service," in *ESEC/FSE*, 2021.

[36] Y. Li, S. Ji, Y. Chen, S. Liang, W. Lee *et al.*, "Unifuzz: A holistic and pragmatic metrics-driven platform for evaluating fuzzers," in *USENIX Security*, 2021.

[37] W. Gao, V.-T. Pham, D. Liu, O. Chang, T. Murray, and B. I. Rubinstein, "Beyond the Coverage Plateau: A Comprehensive Study of Fuzz Blockers (registered report)," in *FUZZING*, 2023, pp. 47–55.

[38] "WebAssembly Runtime Structure," https://webassembly.github.io/spec/core/exec/runtime.html#values, 2025, [Accessed 01-10-2025].

[39] "WebAssembly Change History," https://webassembly.github.io/spec/core/appendix/changes.html, 2025, [Accessed 10-10-2025].

[40] "WASM 2.0," https://webassembly.org/news/2025-03-20-wasm-2.0/, 2025, [Accessed 03-25-2025].

[41] "WASM 3.0," https://webassembly.org/news/2025-09-17-wasm-3.0/, 2025, [Accessed 22-10-2025].

[42] N. Rao, E. Gilbert, H. Green, T. Ramananandro, N. Swamy, C. L. Goues, and S. Fakhoury, "Diffspec: Differential testing with llms using natural language specifications and code artifacts," *arXiv preprint arXiv:2410.04249*, 2024.

[43] Y. Li, Y. Zeng, X. Song, and S. Guo, "Improving Seed Quality with Historical Fuzzing Results," *IST*, vol. 179, p. 107651, 2025.

[44] A. Rebert, S. K. Cha, T. Avgerinos, J. Foote, D. Warren, G. Grieco, and D. Brumley, "Optimizing Seed Selection for Fuzzing," in *USENIX Security*, 2014, pp. 861–875.

[45] A. Herrera, H. Gunadi, S. Magrath, M. Norrish, M. Payer, and A. L. Hosking, "Seed Selection for Successful Fuzzing," in *ISSTA*, 2021, pp. 230–243.

[46] I. Lima, J. Silva, B. Miranda, G. Pinto, and M. d'Amorim, "Exposing bugs in javascript engines through test transplantation and differential testing," *SQJ*, 2021.

[47] S. Zhong and M. Rigger, "Understanding and reusing test suites across database systems," *Proceedings of the ACM on Management of Data*, 2025, sIGMOD.

[48] X. Qin, H. Zhong, H. Yu *et al.*, "Testmig: Migrating gui test cases from ios to android," in *ISSTA*, 2019.

[49] S. Talebipour, Y. Zhao, L. Dojcilović, C. Li, and N. Medvidović, "UI Test Migration across Mobile Platforms," in *ASE*. IEEE, 2021, pp. 756–767.

[50] Y. Zhang, Q. Zhu, J. Yan, C. Liu, W. Zhang, Y. Zhao, D. Hao, and L. Zhang, "Synthesis-based enhancement for gui test case migration," in *ISSTA*, 2024, pp. 869–881.

[51] Y. Gao, W. Hu *et al.*, "Mut: Human-in-the-loop unit test migration," in *ICSE*, 2024.

[52] E. T. Barr, M. Harman, Y. Jia, A. Marginean, and J. Petke, "Automated software transplantation," in *ISSTA*, 2015, pp. 257–269.

[53] T. Zhang and M. Kim, "Automated transplantation and differential testing for clones," in *ICSE*. IEEE, 2017, pp. 665–676.

[54] B. Jiang, Z. Li, Y. Huang, Z. Zhang, and W. Chan, "Wasmfuzzer: A fuzzer for webassembly virtual machines," in *34th International Conference on Software Engineering and Knowledge Engineering, SEKE 2022*. KSI Research Inc., 2022, pp. 537–542.

[55] P. Qian, X. Ying, J. Wang, L. Liu, L. Zhang, J. Chen, and Q. He, "Freewavm: Enhanced webassembly runtime fuzzing guided by parse tree mutation and snapshot," *ISSTA*, vol. 2, no. ISSTA, pp. 159–181, 2025.

[56] Y. Zhang, N. He, J. Gao, S. Cao, K. Liu, H. Wang, Y. Ma, G. Huang, and X. Liu, "DrWASI: LLM-assisted Differential Testing for WebAssembly System Interface Implementations," *TOSEM*, 2026.

[57] X. Zhang, J. Wang, X. Du, and S. Liu, "Wasmcfuzz: Structure-aware fuzzing for wasm compilers," in *Proceedings of the 2024 ACM/IEEE 4th International Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCriS) and 2024 IEEE/ACM Second International Workshop on Software Vulnerability*, 2024, pp. 1–5.

[58] J. Park, Y. Kim, and I. Yun, "Rgfuzz: Rule-guided fuzzer for webassembly runtimes," in *S&P*. IEEE, 2025, pp. 920–938.

[59] J. Wang, Z. Guo, X. Ying, P. Qian, and Y. Chen, "S*w*fuzz: Structure-sensitive webassembly fuzzing," in *APSEC*. IEEE, 2024, pp. 343–352.

[60] J. Zhuang, H. Jiang, and B. Hua, "Wasmchecker: Effectively detecting webassembly bugs via static program analysis," in *2024 8th International Conference on Electrical, Mechanical and Computer Engineering (ICEMCE)*. IEEE, 2024, pp. 1669–1678.

[61] X. Wu, J. He, L. Huang, C. Fu, and W. Wang, "Wbsan: Webassembly bug detection for sanitization and binary-only fuzzing," in *WWW*, 2025, pp. 3311–3322.

[62] C. Watt, M. Trela, P. Lammich, and F. Märkl, "Wasmref-isabelle: A verified monadic interpreter and industrial fuzzing oracle for webassembly," *PLDI*, vol. 7, no. PLDI, pp. 100–123, 2023.

[63] H. Harnes and D. Morrison, "Sok: analysis techniques for webassembly," *Future Internet*, vol. 16, no. 3, p. 84, 2024.

[64] E. Stanley, "Random testing of the webassembly system interface," 2024.

[65] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in *CCS*, 2018.

[66] A. Arcuri and L. Briand, "A practical guide for using statistical tests to assess randomized algorithms in software engineering," in *ICSE*, 2011.

[67] M. Böhme, L. Szekeres, and J. Metzman, "On the reliability of coverage-based fuzzer benchmarking," in *ICSE*, 2022, pp. 1621–1633.

[68] J. Ounjai, V. Wüstholz, and M. Christakis, "Green fuzzer benchmarking," in *ISSTA*, 2023, pp. 1396–1406.

[69] P. Görz, B. Mathis, E. Güler, K. Hassler, T. Holz, and A. Zeller, "Systematic assessment of fuzzers using mutation analysis," in *USENIX Security*, 2023.

[70] J. Kukucka, L. Pina, P. Ammann, and J. Bell, "An empirical examination of fuzzer mutator performance," in *ISSTA*, 2024.